KU LEUVEN

FACULTEIT INGENIEURSWETENSCHAPPEN

Verslag: Warps, thread divergence and striding

Geavanceerde computerarchitectuur

Robbe Hovaert
Lukas Hoste

Master in de ingenieurswetenschappen
Academiejaar 2024-2025

Context

Het doel van dit labo is om kennis te maken met warps, thread divergence en striding. Warps zorgen voor idle cores en thread divergence voor idle threads. Deze dingen hebben een meetbaar effect op de uitvoeringstijd. Om dit te meten doen we een aantal operaties op een foto. We inverteren deze om vervolgens op het rode kanaal Guassian blurring toe te voegen.

Methodologie

Warps

Om inverting toe te passen wordt elke thread een positie/index in de array toegewezen aan de hand van de block, thread id en de block dimensie: $tid = blockIdx.x * blockDim.x + threadIdx.x$. Om striding toe te voegen wordt een for loop gebruikt waarbij als iterator de eerdere thread index gebruikt wordt, deze wordt geïncrementeerd met het aantal threads ($blockDim * gridDim$) en uitgevoerd tot de iterator groter wordt dan de array grootte.

Thread divergence - RGBRGB...RGB

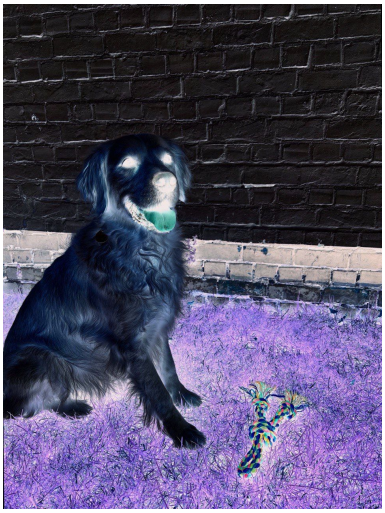
In de eerste versie voor het toe passen van Gaussian blurring op het rode kanaal wordt elke thread toegewezen aan een kleur op basis van zijn thread id. $(tid - 0)\%3$ = rood, $(tid - 1)\%3$ = groen, $(tid - 2)\%3$ = blauw. De threads van het rode kanaal voeren de complexe berekening uit en de andere threads voeren de simpele berekening uit. Hierbij wordt ook striding toegepast op de zelfde manier als bij de inverting kernel.

Thread divergence wegwerken - RRR...GGG...BBB

In de tweede versie voor het toepassen van gaussian blurring op het rode kanaal wordt eerst een transformatie uitgevoeerd op de foto. Hierin worden de R waarden eerst geplaatst, dan alle G waarden en ten laatste alle B waarden. Op deze manier kunnen de startposities van de verschillende kleuren berekend worden. Zo kan aan de hand van de thread id opnieuw gekeken worden welke kleur afgehandeld wordt en welke berekening dus uitgevoerd moet worden. Ten laatste wordt de transformatie terug ongedaan gemaakt. Opnieuw wordt striding op dezelfde manier toegepast. Deze manier zorgt ervoor dat de meeste threads die de kleur rood verwerken (en dus ook de dure operatie uitvoeren) tezamen gescheduled worden, wat er dan voor zorgt dat de meeste niet-zware operaties niet tezamen gescheduled worden met zware operaties.

Observaties en resultaten

Wij hebben aan de bovenstaande implementatie timing-informatie toegevoegd. Wij houden de tijd dat de GPU nodig heeft om dit uit te voeren (tijd zonder memcpy) bij (GPU Time) en de tijd op de CPU om de data te verplaatsen en de operatie op de gpu uit te voeren (GPU CPU Time).



((a)) Inverted



((b)) Inverted + gaussian blur

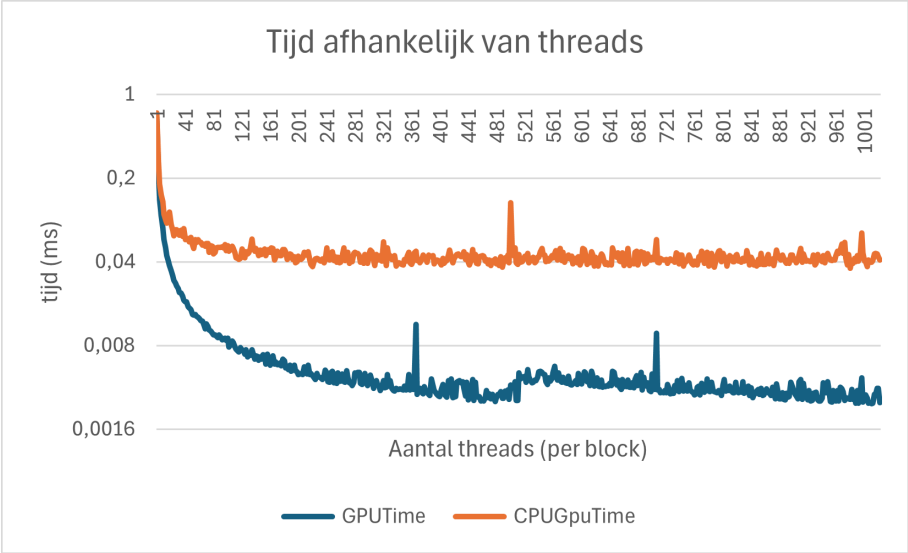
Figuur 1: Inverted foto en foto met gaussian blur

Ten eerste hebben we het effect van het aantal threads op de inverting kernel getest. Hiervoor werd het aantal threads per block gevarieerd van 1 tot 1024 (met 32 blocks). Hierin zijn de verwachte resultaten te zien in Figuur 2, de uitvoeringstijd daalt sterk tot er 32 threads per block aanwezig zijn. Dit is omdat er dan een volledige warp kan geütiliseerd worden. Hierna blijft de performantie vrij gelijk met een zeer kleine variatie mogelijks door het aantal idle cores, GPU temperatuur, ...

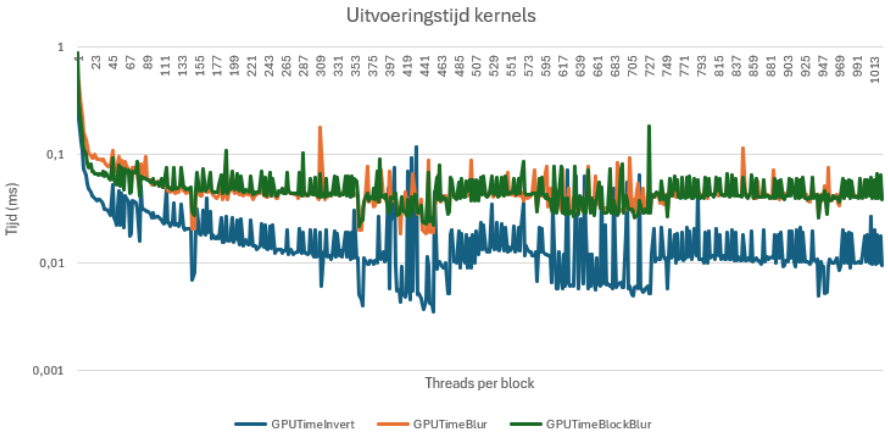
Als tweede werd het verschil in uitvoeringstijd van de verschillende kernels bekeken. Zoals te zien in Figuur 2 zorgt de moeilijkere berekening op het rode kanaal voor een langere uitvoeringstijd. De versie die thread divergence wegwerkt (BlockBlur) toont meestal een iets betere tijd, de verbetering is echter niet bijzonder groot van zodra er voldoende threads aanwezig zijn. Wanneer er minder threads aanwezig zijn is de versie die rekening houd met thread divergence wel beter. Wat logisch is omdat als er voldoende threads zijn het minder erg is dat er enkele threads inactief zijn.

Conlcusie

In dit labo hebben wij kennis gemaakt met warps, thread divergence en striding. Striding zorgt ervoor dat threads in meerdere iteraties kunnen werken, dit zorgt ervoor dat er minder threads dan elementen kunnen zijn zonder dat de functionaliteit niet langer werkt. Warps zorgen ervoor dat het aantal threads slim moet gekozen worden om niet te zorgen voor inactieve threads, dit is makkelijk te zien doordat performantie stabiliseerd van zodra er meer dan 32 threads zijn. Thread divergence is belangrijk om over na te denken wanneer er niet veel threads aanwezig zijn en deze dus ook allemaal zo efficiënt mogelijk moeten benut worden. Met veel threads is het effect ervan minder goed te voelen, maar het blijft toch belangrijk om hier rekening mee te houden.



Figuur 2: Block size tegenover executietijd in ms (gemiddelde van 50 uitvoeringen)



Figuur 3: Block size tegenover executietijd in ms van alle kernels (gemiddelde van 50 uitvoeringen)

A CODE

```
/*
 * Code snippet for importing / exporting image data.
 *
 * To convert an image to a pixel map, run `convert <name>.<extension>
 ↪  <name>.ppm
 *
 */
#include <cstdint>      // Data types
#include <iostream>     // File operations
#include <chrono>

// #define M 512       // Lenna width
// #define N 512       // Lenna height
#define M 960       // VR width
#define N 1280       // VR height
#define C 3         // Colors
#define OFFSET 16   // Header length

#define ARR_SIZE M * N * C


uint8_t* get_image_array(void){
    /*
     * Get the data of an (RGB) image as a 1D array.
     *
     * Returns: Flattened image array.
     *
     * Noets:
     *  - Images data is flattened per color, column, row.
     *  - The first 3 data elements are the RGB components
     *  - The first 3*M data elements represent the firts row of the
     ↪  image
     *  - For example, r_{0,0}, g_{0,0}, b_{0,0}, ..., b_{0,M}, r_{1,0},
     ↪  ..., b_{b,M}, ..., b_{N,M}
     *
     */
    // Try opening the file
    FILE *imageFile;
    imageFile=fopen("./anna.ppm","rb");
    if(imageFile==NULL){
        perror("ERROR: Cannot open output file");
        exit(EXIT_FAILURE);
    }

    // Initialize empty image array
```

```
    uint8_t* image_array =
    ↪  (uint8_t*)malloc(M*N*C*sizeof(uint8_t)+OFFSET);

    // Read the image
    fread(image_array, sizeof(uint8_t), M*N*C*sizeof(uint8_t)+OFFSET,
    ↪  imageFile);

    // Close the file
    fclose(imageFile);

    // Move the starting pointer and return the flattened image array
    return image_array + OFFSET;
}


void save_image_array(uint8_t* image_array){
    /*
     * Save the data of an (RGB) image as a pixel map.
     *
     * Parameters:
     *  - param1: The data of an (RGB) image as a 1D array
     *
     */
    // Try opening the file
    FILE *imageFile;
    imageFile=fopen("./output_image.ppm","wb");
    if(imageFile==NULL){
        perror("ERROR: Cannot open output file");
        exit(EXIT_FAILURE);
    }

    // Configure the file
    fprintf(imageFile,"P6\n");               // P6 filetype
    fprintf(imageFile,"%d %d\n", M, N);      // dimensions
    fprintf(imageFile,"255\n");              // Max pixel

    // Write the image
    fwrite(image_array, 1, M*N*C, imageFile);

    // Close the file
    fclose(imageFile);
}

__global__ void __invertImgGPU(uint8_t* src, uint8_t* dst, int n) {
    for (int tid = blockIdx.x * blockDim.x + threadIdx.x;
        tid < n;
        tid += blockDim.x * gridDim.x)
     {
         dst[tid] = 255 - src[tid];
```

```
    }
}


void invertImgGPU(uint8_t* src, uint8_t* dst, int n, int blockSize, int
↪  amountOfBlocks, float * gpuTime, float * gpuCpuTime) {
    cudaEvent_t start, stop;
    cudaEventCreate(&start);
    cudaEventCreate(&stop);
    const auto startCPU = std::chrono::steady_clock::now();

        // Allocate memory on device
    uint8_t *gpuSrc;
    uint8_t *gpuDst;
    cudaMalloc(&gpuSrc, n * sizeof(uint8_t));
    cudaMalloc(&gpuDst, n *sizeof(uint8_t));
    cudaMemcpy(gpuSrc, src, n * sizeof(uint8_t), cudaMemcpyHostToDevice);
    cudaEventRecord(start);
    __invertImgGPU<<<amountOfBlocks, blockSize>>>(gpuSrc, gpuDst, n);
    cudaDeviceSynchronize(); // Ensure kernel execution completes
    cudaEventRecord(stop);
    cudaEventSynchronize(stop);
    cudaMemcpy(dst, gpuDst, n * sizeof(uint8_t), cudaMemcpyDeviceToHost);
    cudaFree(gpuSrc);
    cudaFree(gpuDst);

    const auto endCPU = std::chrono::steady_clock::now();
    float milliseconds = 0;
    cudaEventElapsedTime(&milliseconds, start, stop);

    *gpuCpuTime = std::chrono::duration<double, std::milli>(endCPU -
    ↪  startCPU).count();
    *gpuTime = milliseconds;

}

__global__ void __blurImgGPU(uint8_t* src, uint8_t* dst, int n) {
    for (int tid = blockIdx.x * blockDim.x + threadIdx.x;
        tid < n;
        tid += blockDim.x * gridDim.x)
     {
         if ((tid - 0) % 3 == 0) {
            int rMinTwo = tid - 2 * 3;
            int rMinOne = tid - 1 * 3;
            int r = tid;
            int rPlusOne = tid + 1 * 3;
            int rPlusTwo = tid + 2 * 3;

            rMinTwo = max(0, rMinTwo);
```

```
            rMinOne = max(0, rMinTwo);
            rPlusOne = min(rPlusOne, (n - (n % 3)));
            rPlusTwo = min(rPlusOne, (n - (n % 3)));
            dst[tid] = 0.1 * src[rMinTwo] + 0.25 * src[rMinOne] + 0.5 *
            ↪  src[r] + 0.25 * src[rPlusOne] + 0.1 * src[rPlusTwo];
        } else if ((tid - 1) % 3 == 0) {
            // green
            dst[tid] = 255 - src[tid];

        } else if ((tid - 2) % 3 == 0) {
            // blue
            dst[tid] = 255 - src[tid];
        }
    }
}

void blurImgGPU(uint8_t* src, uint8_t* dst, int n, int blockSize, int
↪  amountOfBlocks, float * gpuTime, float * gpuCpuTime) {
    cudaEvent_t start, stop;
    cudaEventCreate(&start);
    cudaEventCreate(&stop);
    const auto startCPU = std::chrono::steady_clock::now();
    // Allocate memory on device
    uint8_t *gpuSrc;
    uint8_t *gpuDst;
    cudaMalloc(&gpuSrc, n * sizeof(uint8_t));
    cudaMalloc(&gpuDst, n *sizeof(uint8_t));
    cudaMemcpy(gpuSrc, src, n * sizeof(uint8_t), cudaMemcpyHostToDevice);
    cudaEventRecord(start);
    __blurImgGPU<<<amountOfBlocks, blockSize>>>(gpuSrc, gpuDst, n);
    cudaDeviceSynchronize(); // Ensure kernel execution completes
    cudaEventRecord(stop);
    cudaEventSynchronize(stop);
    cudaMemcpy(dst, gpuDst, n * sizeof(uint8_t), cudaMemcpyDeviceToHost);
    cudaFree(gpuSrc);
    cudaFree(gpuDst);
    const auto endCPU = std::chrono::steady_clock::now();
    float milliseconds = 0;
    cudaEventElapsedTime(&milliseconds, start, stop);

    *gpuCpuTime = std::chrono::duration<double, std::milli>(endCPU -
    ↪  startCPU).count();
    *gpuTime = milliseconds;

}


__global__ void __blockBlurImgGpu(uint8_t* src, uint8_t* dst, int n) {
    // each block is either
```

```
    int colorSize = n / 3;
    int startRed = 0;
    int startGreen = colorSize;
    int startBlue = 2 * colorSize;
    for (int tid = blockIdx.x * blockDim.x + threadIdx.x;
        tid < n;
        tid += blockDim.x * gridDim.x)
     {
        if (tid < startGreen) {
            // the red part
            int rMinTwo = tid - 2;
            int rMinOne = tid - 1;
            int r = tid;
            int rPlusOne = tid + 1;
            int rPlusTwo = tid + 2;
            rMinTwo = max(0, rMinTwo);
            rMinOne = max(0, rMinTwo);
            rPlusOne = min(rPlusOne, startRed);
            rPlusTwo = min(rPlusOne, startGreen);
            dst[tid] = 0.1 * src[rMinTwo] + 0.25 * src[rMinOne] + 0.5 *
            ↪  src[r] + 0.25 * src[rPlusOne] + 0.1 * src[rPlusTwo];
        } else {
            // green, blue
            dst[tid] = 255 - src[tid];
        }

     }
}

void toBlockRGB(uint8_t* src, uint8_t* dst, int n) {
    // on CPU, convert to RRRRRRRRRGGGGGGGGGBBBBBBBBB
    // first all the red values
    int indexSrc = 0;
    int indexDst = 0;
    while (indexSrc < n) {
        dst[indexDst] = src[indexSrc];
        indexSrc += 3;
        indexDst++;
    }
    // then all the green values
    indexSrc = 1;
    while (indexSrc < n) {
        dst[indexDst] = src[indexSrc];
        indexSrc += 3;
        indexDst++;
    }
    // then all the blue values
    indexSrc = 2;
    while (indexSrc < n) {
```

```
        dst[indexDst] = src[indexSrc];
        indexSrc += 3;
        indexDst++;
    }
}

void fromBlockRgb(uint8_t* src, uint8_t* dst, int n) {
    // Convert back to RGBRGB
    int indexSrc = 0;
    int indexDst = 0;

    // Process the red values
    while (indexSrc < n / 3) {
        dst[indexDst] = src[indexSrc];
        indexSrc++;
        indexDst += 3;  // Move to the next RGB position
    }

    // Process the green values
    indexSrc = n / 3;  // Start from the green block
    indexDst = 1;
    while (indexSrc < 2 * n / 3) {
        dst[indexDst] = src[indexSrc];
        indexSrc++;
        indexDst += 3;  // Move to the next RGB position
    }

    // Process the blue values
    indexSrc = 2 * n / 3;  // Start from the blue block
    indexDst = 2;
    while (indexSrc < n) {
        dst[indexDst] = src[indexSrc];
        indexSrc++;
        indexDst += 3;  // Move to the next RGB position
    }
}

void blockBlurImgGPU(uint8_t* src, uint8_t* dst, int n, int blockSize,
↪   int amountOfBlocks, float * gpuTime, float * gpuCpuTime) {
    cudaEvent_t start, stop;
    cudaEventCreate(&start);
    cudaEventCreate(&stop);
    // Allocate memory on device
    uint8_t* blockSrc = (uint8_t*)malloc(n * sizeof(uint8_t));
    uint8_t* blockDst = (uint8_t*)malloc(n * sizeof(uint8_t));
    toBlockRGB(src, blockSrc, n);
    const auto startCPU = std::chrono::steady_clock::now();
    uint8_t *gpuSrc;
    uint8_t *gpuDst;
```

```
    cudaMalloc(&gpuSrc, n * sizeof(uint8_t));
    cudaMalloc(&gpuDst, n *sizeof(uint8_t));
    cudaEventRecord(start);
    cudaMemcpy(gpuSrc, blockSrc, n * sizeof(uint8_t),
    ↪  cudaMemcpyHostToDevice);
    __blockBlurImgGpu<<<amountOfBlocks, blockSize>>>(gpuSrc, gpuDst, n);
    cudaDeviceSynchronize(); // Ensure kernel execution completes
    cudaEventRecord(stop);
    cudaEventSynchronize(stop);
    cudaMemcpy(blockDst, gpuDst, n * sizeof(uint8_t),
    ↪  cudaMemcpyDeviceToHost);
    cudaFree(gpuSrc);
    cudaFree(gpuDst);
    const auto endCPU = std::chrono::steady_clock::now();
    float milliseconds = 0;
    cudaEventElapsedTime(&milliseconds, start, stop);

    *gpuCpuTime = std::chrono::duration<double, std::milli>(endCPU -
    ↪  startCPU).count();
    *gpuTime = milliseconds;
    fromBlockRgb(blockDst, dst, n);
    free(blockSrc);
    free(blockDst);
    return;

}




int main (void) {

    // Read the image
    uint8_t* image_array = get_image_array();

    // Allocate output
    uint8_t* new_image_array = (uint8_t*)malloc(M*N*C);

    /*// Convert to grayscale using only the red color component
    for(int i=0; i<M*N*C; i++){
        new_image_array[i] = image_array[i/3*3];
    }*/

    // execute once to warm up
    //invertImgGPU(image_array, new_image_array, ARR_SIZE, 1, 16,
    ↪  &gpuInvertTime, &gpuCpuInvertTime);
    // invoke the inverse kernel from loop, gradually increase the amount
    ↪  of threads
```

```
    // we do this to see the effect of the warps
    // per amount of threads do 100 iterations and take an average
    printf("ThreadsPerBlock, GPUTimeInvert, CPUGPUimeInvert, GPUTimeBlur,
    ↪  CPUGPUimeBlur, GPUTimeBlockBlur, CPUGPUTimeBlockBlur , Warps,
    ↪  IdleCores, UsedCores\n");
    for (int i = 1; i <= 1024; i+=2) {

        float gpuInvertTime = 0;
        float gpuCpuInvertTime = 0;
        float gpuBlurTime = 0;
        float gpuCpuBlurTime = 0;
        float gpuBlockBlurTime = 0;
        float gpuCpuBlockBlurTime = 0;

        // calculate the number of warps: threads per block / 32 rounded
        ↪  up * number of blocks
        int warps = ceil(i / 32.0) * 32;
        // calculate how many cores will be idle -> warps*32 - threads
        ↪  per block*number of blocks
        int idleCores = warps * 32 - i * 32;
        int usedCores = i * 32;

        for (int j = 0; j < 50; j++) {
            invertImgGPU(image_array, new_image_array, ARR_SIZE, i, 32,
            ↪  &gpuInvertTime, &gpuCpuInvertTime);
            blurImgGPU(image_array, new_image_array, ARR_SIZE, i, 32,
            ↪  &gpuBlurTime, &gpuCpuBlurTime);
            blockBlurImgGPU(image_array, new_image_array, ARR_SIZE, i,
            ↪  32, &gpuBlockBlurTime, &gpuCpuBlockBlurTime);
        }
        printf("%d; %f; %f; %f; %f; %f; %f; %d; %d; %d\n", i,
        ↪  gpuInvertTime / 50, gpuCpuInvertTime / 50, gpuBlurTime / 50,
        ↪  gpuCpuBlurTime / 50, gpuBlockBlurTime / 50,
        ↪  gpuCpuBlockBlurTime / 50, warps, idleCores, usedCores);
        //printf("%d, %f, %f, %d, %d, %d\n", i, gpuTime / 50, gpuCpuTime
        ↪  / 50, warps, idleCores, usedCores);
    }


//    blurImgGPU(image_array, new_image_array, ARR_SIZE, 2, 16);
   //invertImgGPU(image_array, new_image_array, ARR_SIZE ,1, 16);

    // Save the image
   //blurImgGPU(image_array, new_image_array, ARR_SIZE, 2, 16);
   //blockBlurImgGPU(image_array, new_image_array, ARR_SIZE, 2, 16);
    //save_image_array(new_image_array);
    //printf("Done!\n");
    //printf("%d", sizeof(uint8_t));
    free(new_image_array);
```

```
    return 0;
}
```